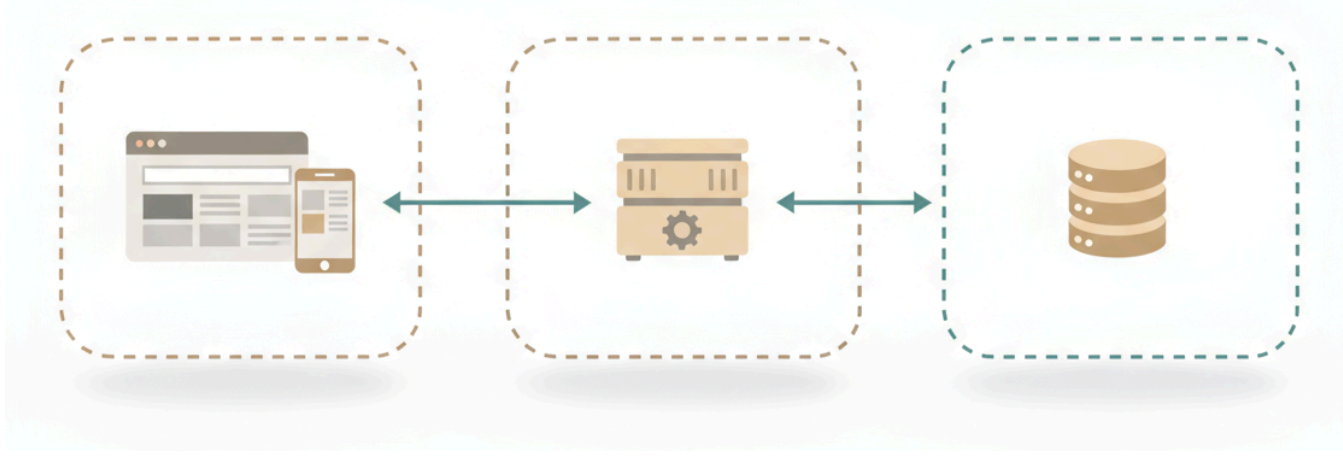
Datengewinnung & -aufbereitung

2. Datenbanken und SQL

Markus Schanta



Architektur einer Webapplikation



Komponenten einer Datenbank

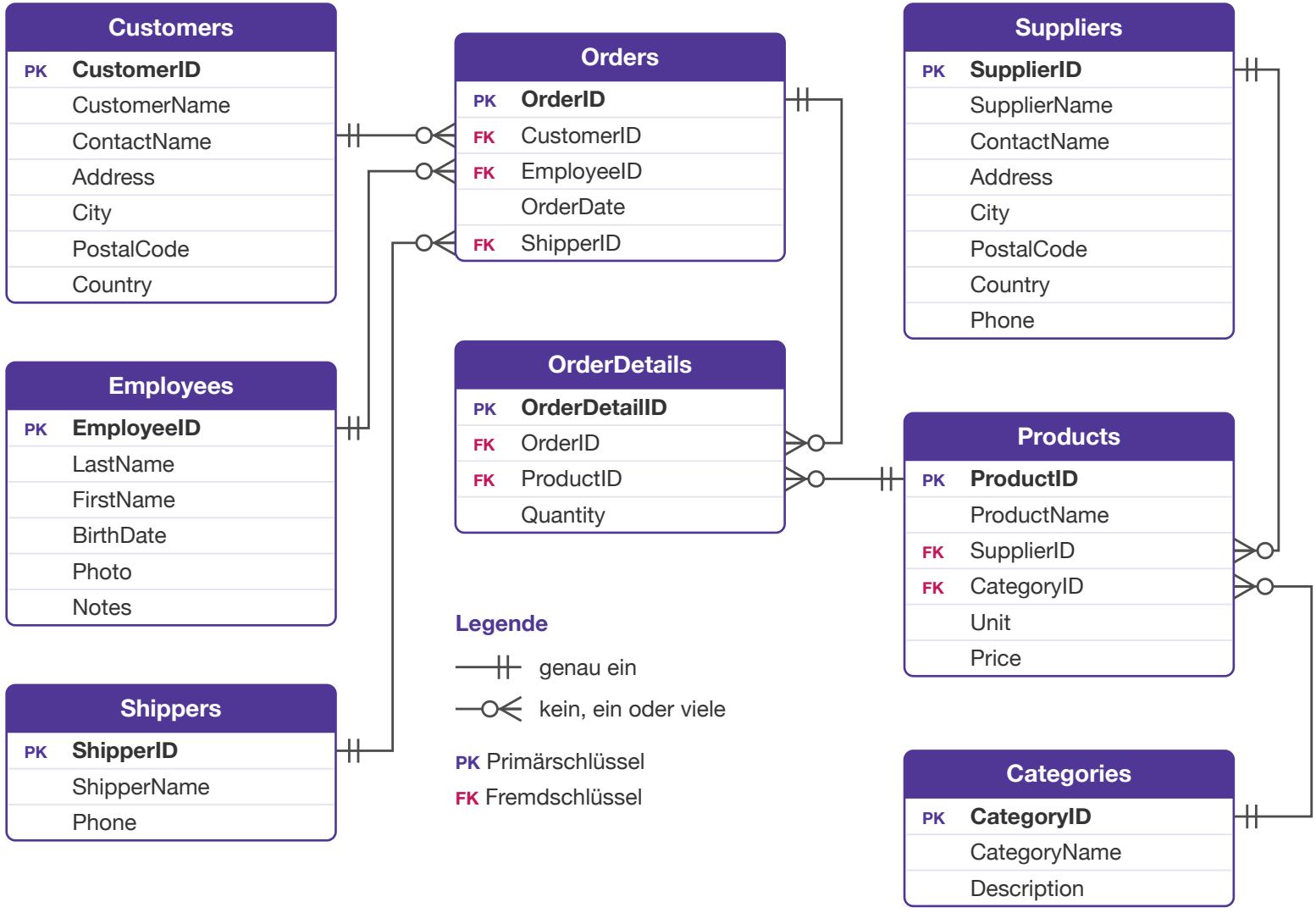


Aufbau einer Tabelle

Number of Records: 77

ProductID	ProductName	SupplierID	CategoryID	Unit	Price
1	Chais	1	1	10 boxes x 20 bags	18
2	Chang	1	1	24 - 12 oz bottles	19
3	Aniseed Syrup	1	2	12 - 550 ml bottles	10
4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22
5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.35

Entity-Relationship-Diagramm



Der Begriff „Datenbank“

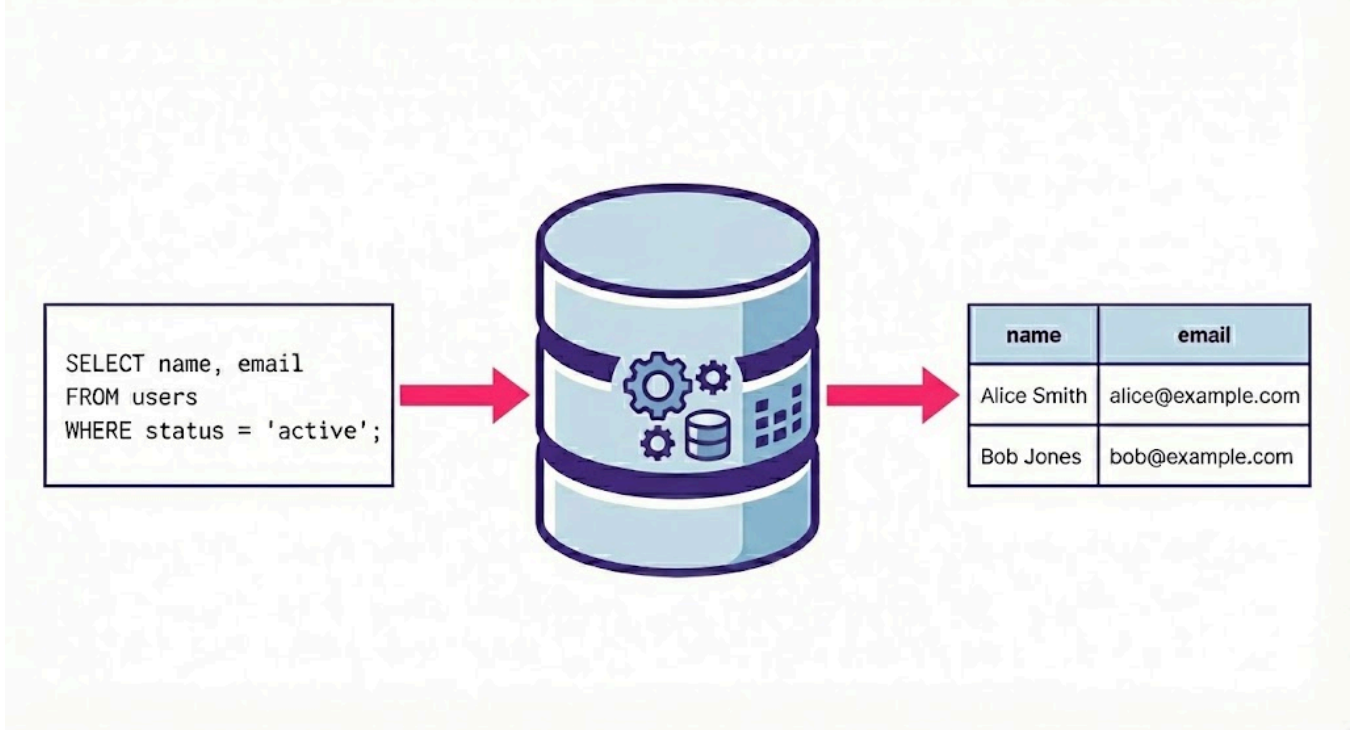
bezeichnet

1. eine strukturierte Sammlung von Daten (= Schema + Tabelleninhalte) und
2. Mechanismen und Protokolle zur Verwaltung dieser Daten.

SQL

- **SQL** ist ein Akronym für **S**tructured **Q**uery **L**anguage.
- Erstmals 1974 veröffentlicht,
  seit 1986 ANSI- und seit 1987 ISO-Standard.
- Vier Hauptfunktionen:
    1. Create (Schlüsselwort `INSERT`)
    2. **Retrieve** (Schlüsselwort `SELECT`)
    3. Update (Schlüsselwort `UPDATE`)
    4. Delete (Schlüsselwort `DELETE`)

Request → Response



SELECT-Abfragen

- Grundstruktur einer SQL-Abfrage:

```
SELECT spalte1, spalte2, ...
FROM tabellenname
```

- **SELECT** gibt an, welche Spalten ausgewählt werden
- **FROM** gibt an, aus welcher Tabelle die Daten stammen
- Spalte `CustomerName` aus der Tabelle `Customers` auswählen:

```
SELECT CustomerName
FROM Customers
```

- Alle Spalten aus der Tabelle `Customers` auswählen:

```
SELECT *
FROM Customers
```

SQL-Editor

Interaktiver SQL-Editor auf der Kursseite:
markusschanta.github.io/fhwien-dga-2026/sql-editor/

SELECT Beispiel 1

- Die Spalte `ContactName` aus der Tabelle `Customers` auswählen:

```
SELECT ContactName
FROM Customers
```

SELECT Beispiel 2

- Die Postleitzahl und Stadt aller Kunden ausgeben:

```
SELECT PostalCode, City
FROM Customers
```

Abfragen mit WHERE-Bedingungen

- Grundlegende Struktur einer **SELECT**-Abfrage mit **WHERE**-Filter:

```
SELECT spalte1, spalte2, ...
FROM tabellenname
WHERE bedingung1
    AND/OR bedingung2
    AND/OR ...;
```

Spalten / Zeilen

- **SELECT** bestimmt die **Spalten**, **WHERE** die **Zeilen**:

```
SELECT spalte1, spalte2, ...
FROM tabellenname
WHERE bedingung
```



Result:

Number of Records: 91

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden
6	Blauer See Delikatessen	Hanna Moos	Forsterstr. 57	Mannheim	68306	Germany

Operatoren für `WHERE`-Bedingungen

Operator	Bedingung	Beispiel
`=, !=, <, <=, >, >=`	Vergleichsoperatoren	`spalte > 4`
`BETWEEN ... AND ...`	Wert zwischen zwei Werten	`spalte BETWEEN 1.5 AND 10.5`
`NOT BETWEEN ... AND ...`	Wert nicht zwischen Werten	`spalte NOT BETWEEN 1 AND 10`
`IN (...)`	Wert ist in einer Liste	`spalte IN (2, 4, 6)`
`NOT IN (...)`	Wert ist nicht in einer Liste	`spalte NOT IN (1, 3, 5)`

WHERE-Bedingungen Beispiel 1

- Namen und Preise aller Produkte mit einem Preis von mehr als 50 ausgeben:

```
SELECT ProductName, Price
FROM Products
WHERE Price > 50
```

`WHERE`-Bedingungen Beispiel 2

- `Namen` aller `Produkte` mit
  einem `Preis` zwischen `10` und `13` ausgeben:

```
SELECT ProductName
FROM Products
WHERE Price BETWEEN 10 AND 13
```

- *alternativ*

```
SELECT ProductName
FROM Products
WHERE Price >= 10 AND Price <= 13
```

Übung: `WHERE` 1/4

Name und Preis aller Produkte unter 10 ausgeben

Interaktiver SQL-Editor auf der Kursseite:
markusschanta.github.io/fhwien-dga-2026/sql-editor/

WHERE: Operatoren für Text

Operator	Bedingung	Beispiel
`=`	Exakte Übereinstimmung	`spalte = 'abc'`
`!=` oder `<>`	Ungleichheit	`spalte != 'abcd'`
`LIKE`	Übereinstimmung mit Muster	`spalte LIKE 'ABC'`
`NOT LIKE`	Keine Übereinstimmung mit Muster	`spalte NOT LIKE 'ABCD'`
`%`	Platzhalter für mehrere (oder keine) Zeichen	`spalte LIKE '%AT%'` (findet 'AT', 'ATTIC', 'CAT' oder 'BATS')
`_`	Platzhalter für genau ein Zeichen	`spalte LIKE 'AN_'` (findet 'AND', aber nicht 'AN')

Texte stehen in einfachen Anführungszeichen.
Ob Groß- und Kleinschreibung beachtet wird, hängt vom Datenbanksystem ab.

WHERE-Bedingungen Beispiel 3

- Alle Kunden aus Deutschland ausgeben:

```
SELECT *
FROM Customers
WHERE Country = 'Germany'
```

Übung: `WHERE` 2/4

Alle Kunden aus London ausgeben

Interaktiver SQL-Editor auf der Kursseite:
markusschanta.github.io/fhwien-dga-2026/sql-editor/

Mehrere Bedingungen verknüpfen

Operator	Bedeutung	Beispiel
`AND`	Beide Bedingungen sind erfüllt	`Country = 'Germany' AND City = 'Berlin'`
`OR`	Mindestens eine ist erfüllt	`Country = 'Germany' OR Country = 'Austria'`
`NOT`	Bedingung ist nicht erfüllt	`NOT Country = 'Germany'`

OR Beispiel

- Alle Kunden aus Deutschland oder Österreich ausgeben:

```
SELECT CustomerName, City, Country
FROM Customers
WHERE Country = 'Germany' OR Country = 'Austria'
```

Übung: `WHERE` 3/4

Alle Kunden aus Spanien, Frankreich oder Italien ausgeben

Interaktiver SQL-Editor auf der Kursseite:
markusschanta.github.io/fhwien-dga-2026/sql-editor/

WHERE-Bedingungen Beispiel 4

- Alle Kunden aus Ländern ausgeben, deren Name auf „land“ endet:

```
SELECT *
FROM Customers
WHERE Country LIKE '%land'
```

WHERE-Bedingungen Beispiel 5

- Alle Kunden mit einer vierstelligen `Postleitzahl` ausgeben:

```
SELECT *
FROM Customers
WHERE PostalCode LIKE '____'
```

Übung: WHERE 4/4

Alle Kunden aus Deutschland ausgeben, deren Name mit „B“ beginnt

Interaktiver SQL-Editor auf der Kursseite:
markusschanta.github.io/fhwien-dga-2026/sql-editor/

Sortierung mit `ORDER BY`

- Sortiert das Ergebnis nach der angegebenen Spalte:

```
SELECT spalte1, spalte2, ...
FROM tabellenname
WHERE bedingung
ORDER BY spalte1 [ASC|DESC]
```

- Standardmäßig aufsteigend (**`ASC`**)
- Absteigend mit **`DESC`**

`ORDER BY` Beispiel 1

- Alle `Produkte` sortiert nach `Preis` ausgeben:

```
SELECT *
FROM Products
ORDER BY Price ASC
```

`ORDER BY` Beispiel 2

- Die 5 teuersten `Produkte` ausgeben:

```
SELECT TOP 5 ProductName, Price
FROM Products
ORDER BY Price DESC
```

- **`TOP`** begrenzt die Anzahl der Zeilen im Ergebnis
- In anderen Datenbanksystemen (z. B. MySQL, SQLite): **`LIMIT 5`** am Ende der Abfrage

Übung zum Abschluss 1/3

Name und Preis aller Produkte der Kategorie 1 ausgeben, die teuersten zuerst

Interaktiver SQL-Editor auf der Kursseite:
markusschanta.github.io/fhwien-dga-2026/sql-editor/

Übung zum Abschluss 2/3

Die 3 günstigsten Produkte ausgeben, die mehr als 20 kosten

Interaktiver SQL-Editor auf der Kursseite:
markusschanta.github.io/fhwien-dga-2026/sql-editor/

Übung zum Abschluss 3/3

Name und Stadt aller Kunden aus den USA ausgeben, alphabetisch nach Stadt sortiert

Interaktiver SQL-Editor auf der Kursseite:
markusschanta.github.io/fhwien-dga-2026/sql-editor/

Fragen?

Nächste Schritte

- Gruppeneinteilung in Moodle
- Folien auf der Kursseite (Link in Moodle)
- Beispiele und Übungen im SQL-Editor wiederholen (Link auf der Kursseite)
- Für Teil 3 (Python): Google-Konto erstellen

Danke und schönen Abend!

Rückblick

```sql
SELECT ProductName, Price
FROM Products
WHERE Price > 20
ORDER BY Price DESC
```

- **SELECT**: welche Spalten
- **FROM**: aus welcher Tabelle
- **WHERE**: welche Zeilen
- **ORDER BY**: in welcher Reihenfolge

De-Duplizierung mit `DISTINCT`

- Entfernt doppelte Einträge aus dem Ergebnis:

```
SELECT DISTINCT spalte1
FROM tabellenname
WHERE bedingung
```

Gruppierung & Aggregatfunktionen

- Gruppierung mit **GROUP BY**:

```
SELECT CategoryID
FROM Products
GROUP BY CategoryID
```

Gruppierung & Aggregatfunktionen

- Gruppierung mit **`GROUP BY`** und Aggregatfunktion **`COUNT()`**:

```sql
SELECT CategoryID, COUNT(ProductID)
FROM Products
GROUP BY CategoryID
```

Aggregatfunktionen in SQL

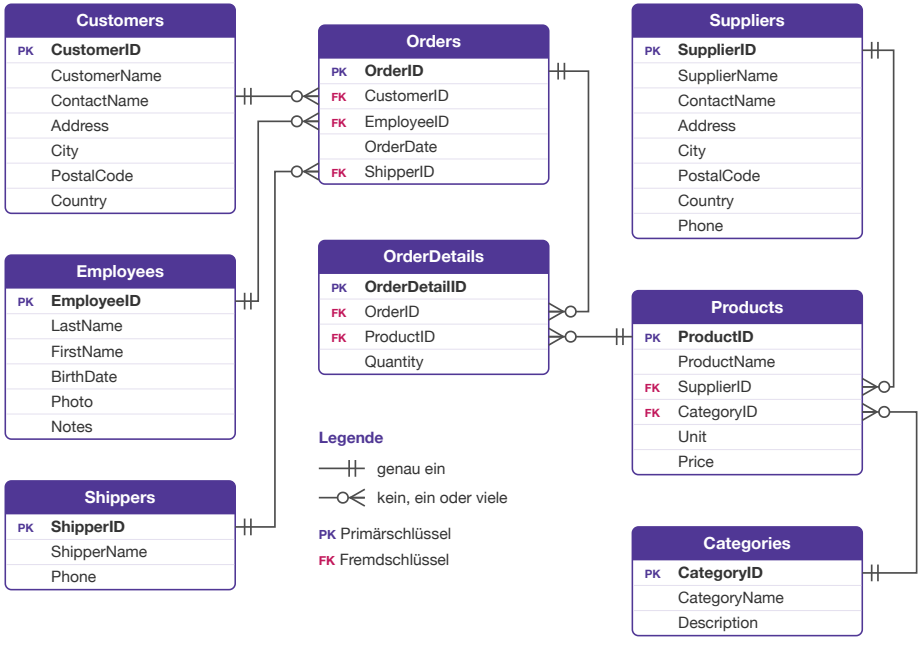
Funktion	Bedeutung
`COUNT(*)`	Anzahl aller Zeilen
`COUNT(spalte)`	Anzahl nicht-null Werte in `spalte`
`MIN(spalte)`	Kleinster Wert in `spalte`. `spalte` muss numerisch sein.
`MAX(spalte)`	Größter Wert in `spalte`. `spalte` muss numerisch sein.
`AVG(spalte)`	Durchschnittlicher Wert in `spalte`. `spalte` muss numerisch sein.
`SUM(spalte)`	Summe aller Werte in `spalte`. `spalte` muss numerisch sein.

GROUP BY Beispiel 1

- Gib den durchschnittlichen `Preis` pro `Produktkategorie` aus:

```
SELECT CategoryID, AVG(Price)
FROM Products
GROUP BY CategoryID
```

Normalisierung



Number of Records: 77

ProductID	ProductName	SupplierID	CategoryID	Unit	Price
1	Chais	1	1	10 boxes x 20 bags	18
2	Chang	1	1	24 - 12 oz bottles	19
3	Aniseed Syrup	1	2	12 - 550 ml bottles	10
4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22
5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.35

Number of Records: 8

CategoryID	CategoryName	Description
1	Beverages	Soft drinks, coffees, teas, beers, and ales
2	Condiments	Sweet and savory sauces, relishes, spreads, and seasonings
3	Confections	Desserts, candies, and sweet breads
4	Dairy Products	Cheeses
5	Grains/Cereals	Breads, crackers, pasta, and cereal
6	Meat/Poultry	Prepared meats
7	Produce	Dried fruit and bean curd
8	Seafood	Seaweed and fish

Joins von Tabellen

- **LEFT JOIN** der Tabellen `Products` und `Categories`:

```
SELECT *
FROM Products
LEFT JOIN Categories
  ON Products.CategoryID = Categories.CategoryID
```

Number of Records: 77

ProductID	ProductName	SupplierID	Products.CategoryID	Unit	Price	Categories.CategoryID	CategoryName	Description
1	Chais	1	1	10 boxes x 20 bags	18	1	Beverages	Soft drinks, coffees, teas, beers, and ales
2	Chang	1	1	24 - 12 oz bottles	19	1	Beverages	Soft drinks, coffees, teas, beers, and ales
3	Aniseed Syrup	1	2	12 - 550 ml bottles	10	2	Condiments	Sweet and savory sauces, relishes, spreads, and seasonings
4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22	2	Condiments	Sweet and savory sauces, relishes, spreads, and

JOIN Beispiel 1

- Gib alle Produkte aus der Produktkategorie Produce aus:

```
SELECT *
FROM Products
LEFT JOIN Categories
  ON Products.CategoryID = Categories.CategoryID
WHERE CategoryName='Produce'
```

Number of Records: 5

ProductID	ProductName	SupplierID	Products.CategoryID	Unit	Price	Categories.CategoryID	CategoryName	Description
14	Tofu	6	7	40 - 100 g pkgs.	23.25	7	Produce	Dried fruit and bean curd
7	Uncle Bob's Organic Dried Pears	3	7	12 - 1 lb pkgs.	30	7	Produce	Dried fruit and bean curd
28	Rössle Sauerkraut	12	7	25 - 825 g cans	45.6	7	Produce	Dried fruit and bean curd
74	Longlife Tofu	4	7	5 kg pkg.	10	7	Produce	Dried fruit and bean curd
51	Manjimup Dried Apples	24	7	50 - 300 g pkgs.	53	7	Produce	Dried fruit and bean curd

JOIN Beispiel 2

- Gib die Anzahl der Produkte pro Produktkategorie aus:

```
SELECT CategoryName, COUNT(ProductID) as ProductIDCount
FROM Products
LEFT JOIN Categories
  ON Products.CategoryID = Categories.CategoryID
GROUP BY CategoryName
```

Number of Records: 8

CategoryName	ProductIDCount
Beverages	12
Condiments	12
Confections	13
Dairy Products	10
Grains/Cereals	7
Meat/Poultry	6
Produce	5
Seafood	12

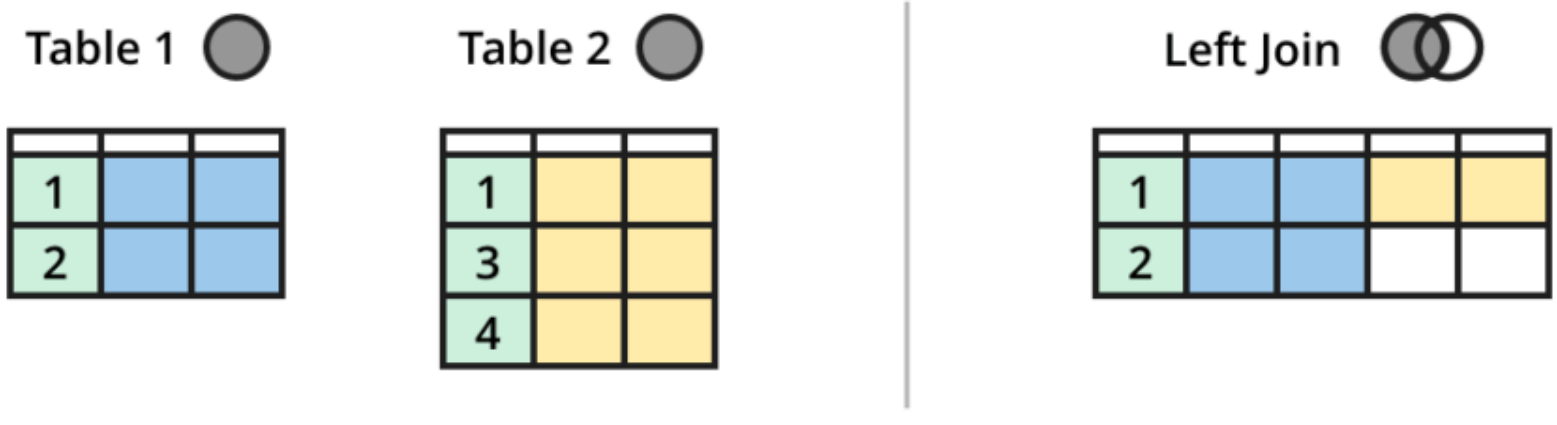
JOIN Beispiel 3

- JOIN über mehrere Tabellen: Gib alle Produkte mit Lieferantenname und Produktkategorie aus:

```
SELECT *
FROM Products p
LEFT JOIN Categories c
  ON p.CategoryID=c.CategoryID
LEFT JOIN Suppliers s
  ON p.SupplierID=s.SupplierID
```

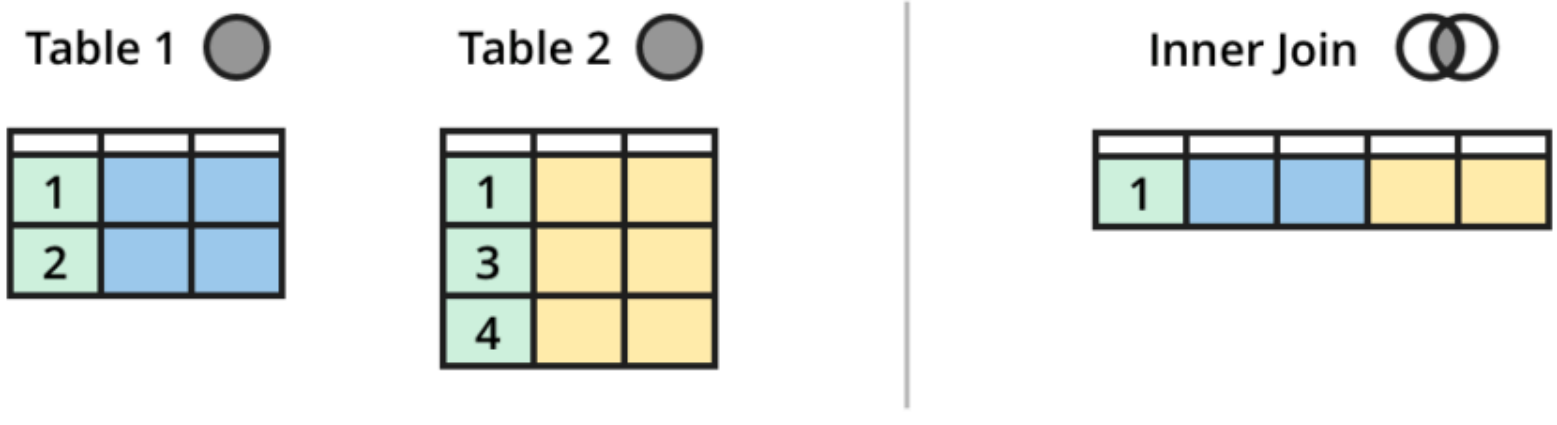
`LEFT JOIN` Bedeutung

`LEFT JOIN` bedeutet, dass alle Zeilen der linken Tabelle im Ergebnis enthalten sind, Zeilen der rechten Tabelle aber nur, wenn es einen passenden Eintrag gibt.



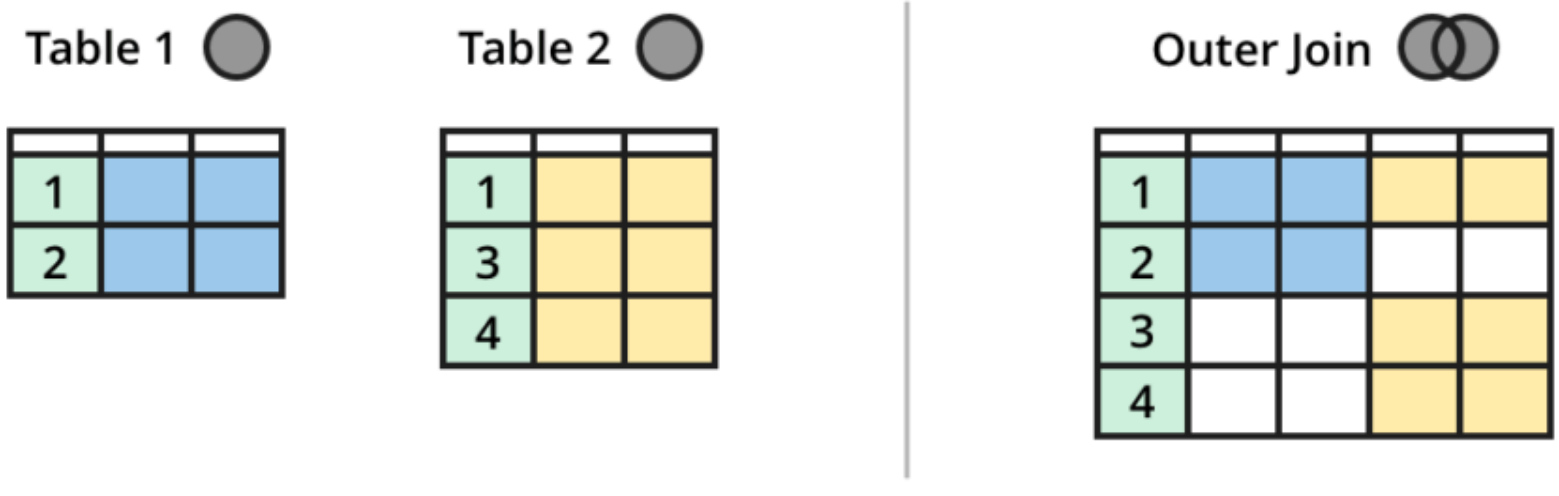
INNER JOIN Bedeutung

INNER JOIN bedeutet, dass ausschließlich, die Zeilen der linken und rechten Tabelle im Ergebnis enthalten sind, die übereinstimmende Werte in den verbundenen Spalten haben.



`OUTER JOIN` Bedeutung

`OUTER JOIN` bedeutet, dass alle Zeilen der linken und rechten Tabelle im Ergebnis enthalten sind und übereinstimmende Werte im Ergebnis in einer Zeile zusammengeführt werden.



Fragen?

Nächste Schritte

- Folien & Materialien der heutigen Einheit auf Moodle
- Nächste Einheit: Do. 4.12. 18:30

Danke und schönen Abend!